

TP — Application de Commande de Repas (TypeScript)

Durée : 4h30

Objectif

Développer une **mini application web en TypeScript** permettant :

- de récupérer des repas depuis une API
- de passer des commandes
- de gérer un portefeuille utilisateur
- de conserver l'historique des commandes

Contraintes :

- Pas de framework
- HTML + TypeScript uniquement
- Le code doit **compiler sans erreur**
- Le code doit être isolé par cas d'usage (un fichier pour l'api, un fichier pour le user, un pour les repas etc.)

Structure du projet

```
/project
  index.html
  /src
    app.ts
    user.ts
    meals.ts
  /dist
    app.js
  tsconfig.json
```

Le fichier HTML est fourni.

Vous devez principalement travailler dans :

```
src/app.ts
```

Données

L'API fournie est disponible à l'adresse suivante : <https://keligmartin.github.io/api/meals.json>

Vous devrez développer cette méthode pour récupérer les repas :

```
fetchMeals(): Promise<Meal[]>
```

L'api retourne :

```
type Meal = {
  id: number
  name: string
  calories: number
  price: number
}
```

Cette API peut **échouer aléatoirement**.

Vous devez donc **gérer les erreurs**.

Étape 1 — Modélisation

Créer les types (ou interfaces) suivants :

```
type Meal = {  
  id: number  
  name: string  
  calories: number  
  price: number  
}  
  
type Order = {  
  id: number  
  meals: Meal[]  
  total: number  
}
```

Créer également une **classe User**.

Elle devra contenir :

```
id  
name  
wallet  
orders
```

Étape 2 — Récupération des repas

Récupérer les repas depuis l'API.

L'API peut échouer.

Vous devez :

- gérer l'erreur avec `try/catch`
- afficher un message dans la console si l'API échoue

Exemple attendu :

```
Erreur lors du chargement des repas
```

Étape 3 — Affichage des repas

Afficher les repas dans la liste HTML :

```
Burger - 10€  
Pizza - 12€  
Salad - 8€
```

Dans l'élément :

```
<ul id="mealList"></ul>
```

Chaque repas doit avoir un **bouton permettant de le commander**.

Étape 4 — Portefeuille utilisateur

L'utilisateur possède un **portefeuille**. Le portefeuille peut être soit un nombre, soit un objet (et l'utilisateur pourrait avoir plusieurs portefeuilles)

Exemple :

```
const user = new User(1, "Bob", 30)
```

L'utilisateur peut commander un repas.

Créer une méthode :

```
orderMeal(meal: Meal)
```

Cette méthode doit :

1. vérifier si l'utilisateur a assez d'argent
2. retirer le prix du portefeuille
3. ajouter la commande dans l'historique

Gestion d'erreur

Si l'utilisateur **n'a pas assez d'argent**, une exception doit être levée.

Exemple :

```
throw new TropPauvreErreur("Fonds insuffisants")
```

Si il y a un peu plus de détails dans l'erreur c'est évidemment mieux

Exemple :

```
- prix restant sur le portefeuille  
- prix total de la commande
```

Étape 5 — Historique des commandes persistant

L'historique des commandes doit être **persistant**.

Vous devez utiliser :

```
localStorage
```

À chaque commande :

- sauvegarder l'historique
- le recharger au démarrage de l'application

Étape 6 — Utility Types

Utiliser au moins **deux Utility Types TypeScript**.

Exemples possibles :

```
Partial  
Omit  
Record
```

Exemple :

```
type MealDraft = Partial<Meal>
```

Étape 7 — Manipulation du DOM

Vous devez manipuler le DOM pour :

- afficher les repas
- afficher le portefeuille utilisateur

- afficher l'historique des commandes

Bonus

Implémenter une ou plusieurs fonctionnalités :

- possibilité de **supprimer une commande**
- afficher **le total dépensé**
- filtrer les repas par prix
- ajouter un **menu contenant plusieurs repas**

Évaluation

Critère	Points
Le code compile sans erreurs	2
Récupération des meals depuis l'API	2
Gestion des erreurs de l'API	2
Portefeuille utilisateur	2
Exception si fonds insuffisants	2
Historique persistant (localStorage)	2
Utilisation d'Utility Types	2
Manipulation du DOM	2
Bonus	4

Total : **20 points**

Conseils

- Travaillez étape par étape
- Testez régulièrement votre code
- Utilisez les types TypeScript au maximum
- Pensez à gérer les erreurs
- Faites des push de temps en temps
- Rendez le repo git dès le début du TP

Rendu

- Rendu par mail à kmartin16@myges.fr
- Sujet : TP Uberscript - NOM Prénom
- Corps du mail : Lien vers le repo git
- Zip à ajouter sur myges au cas où